



bbFMR

Auswertung breitbandiger ferromagnetischer Resonanzmessungen

EINFÜHRUNG IN DAS PROGRAMMIERPROJEKT

Eine Einführung für Mentor / Betreuung

Portierung der LabVIEW-Auswertung nach Python

Autor: Ibrahim Yalcinsoy · Paket `bbfmr`, Version 0.1.0

1 Worum es geht

bbFMR wertet **breitbandige ferromagnetische Resonanzmessungen** (bbFMR) quantitativ aus. Das Programm liest die Rohmessdaten des Messplatzes ein und gewinnt daraus die magnetischen Materialgrößen einer Probe.

Bei einer FMR-Messung wird eine magnetische Probe einem äußeren Magnetfeld ausgesetzt und mit Mikrowellen variabler Frequenz durchstrahlt. Bei einer materialspezifischen Kombination aus Anregungsfrequenz und Magnetfeld absorbiert die Probe resonant Energie. Aus der Lage des Resonanzfeldes als Funktion der Frequenz, $B_{\text{res}}(f)$, sowie aus der frequenzabhängigen Linienbreite lassen sich drei zentrale Materialkenngrößen bestimmen:

Größe	Symbol	Bedeutung
Effektive Magnetisierung	$\mu_0 M_{\text{eff}}$	aus der Dispersion $B_{\text{res}}(f)$ (Kittel)
Landé-Faktor	g	aus der Steigung der Dispersion
Gilbert-Dämpfung	α	aus der Steigung der Linienbreite über f (LLG)

Die Eingangsdaten liegen als **TDMS-Dateien** (LabVIEW-Format) vor; bbFMR überführt sie in physikalische Kenngrößen, Diagramme und tabellarische Exporte (Excel/CSV). Das Projekt ist die **Portierung einer bestehenden LabVIEW-Auswertung nach Python** – mit dem Ziel einer transparenten, prüfbaren und erweiterbaren Auswertekette.

Begriffe in Kürze. *Linescan* – ein Feld-Sweep bei fester Anregungsfrequenz; das komplexe Transmissionssignal S_{21} als Funktion des Feldes. S_{21} – komplexer Streuparameter (Real- und Imaginärteil) des Durchgangs. *Resonanzfeld* B_{res} – Feld, bei dem die Resonanzbedingung erfüllt ist. *Fenster* – der Feldausschnitt um die Resonanz, der in den Fit eingeht.

Was das Programm konkret leistet

- automatisches Einlesen und Erkennen zweier TDMS-Messformate (roh / vorverarbeitet);
- automatische Bestimmung des Resonanzfensters je Frequenz (*AutoWindow*);
- nichtlinearer Fit des komplexen S_{21} -Signals an die Polder-Suszeptibilität;
- automatische Qualitätsbewertung jedes Einzel-Fits (unauffällig / problematisch);

- aggregierte Kittel-/LLG-Auswertung zu $\mu_0 M_{\text{eff}}$, g , α ;
- grafische Oberfläche (PySide6) und programmatische Python-Schnittstelle;
- Export nach Excel/CSV sowie ein Robustheits-Prüfwerkzeug über reale Messreihen.

2 Was bei einer Auswertung passiert

Die beiden folgenden Abbildungen zeigen – mit den echten Modellfunktionen des Programms erzeugt – die zwei Ebenen der Auswertung: den Fit eines einzelnen Linescans und die anschließende Gewinnung der Materialgrößen aus allen Einzel-Fits.

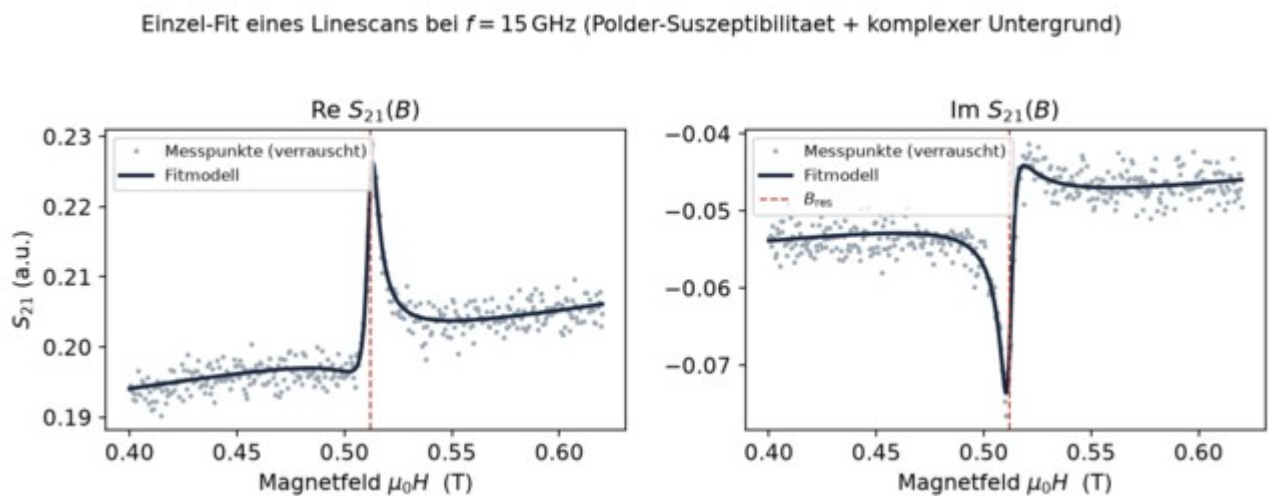


Abb. 1 — Ein Einzel-Fit: Real- und Imaginärteil von S_{21} über dem Magnetfeld bei fester Frequenz. Die schmale dispersive Resonanz sitzt auf einem glatten Untergrund; das Modell (Polder-Suszeptibilität + komplexer Untergrund) wird simultan an Re und Im angepasst. Die rote Linie markiert das Resonanzfeld B_{res} .

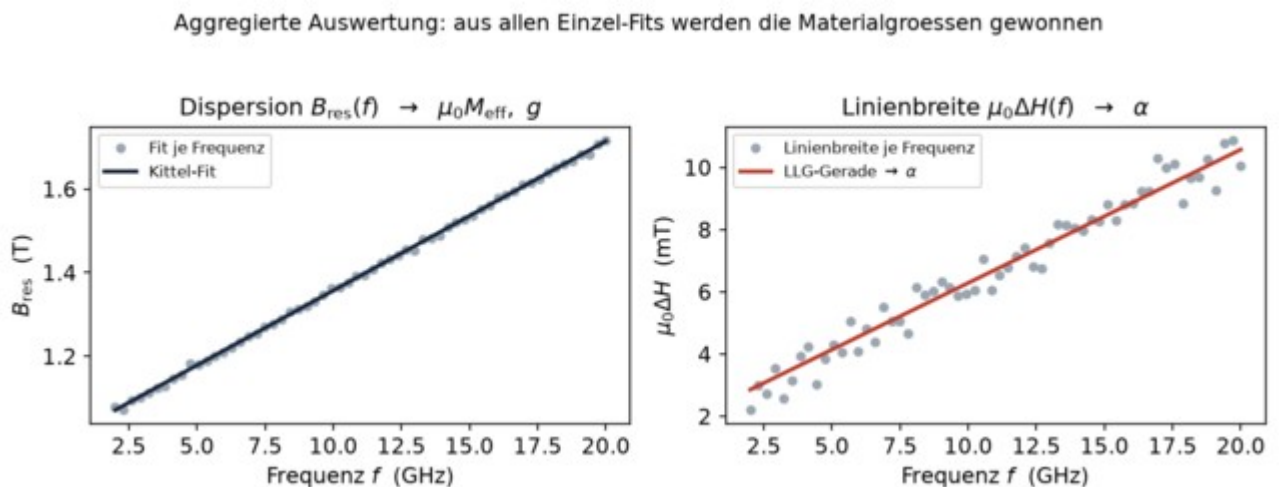


Abb. 2 — Aggregierte Auswertung: Aus den B_{res} -Werten aller Frequenzen ergibt sich über die Kittel-Gleichung die effektive Magnetisierung und der g -Faktor (links); aus der frequenzabhängigen Linienbreite über die LLG-Gerade die Gilbert-Dämpfung α (rechts).

3 Architektur des Codes

Der Quelltext ist nach Verantwortlichkeiten in klar getrennte Module gegliedert. Jeder Schritt der Auswertung ist genau einem Modul zugeordnet:

bbfmr/	
io/	Einlesen/Schreiben TDMS, Datenstruktur (Linescan, Messdatensatz)
physik/	Konstanten, Polder-Suszeptibilität, Fitmodell, Kittel/LLG
fit/	AutoWindow, Einzelfit (lmfit), Stapelverarbeitung, Bewertung
auswertung/	Resonanz vs. f/T , Kittel-/LLG-Fit, Publikationsplots
persistenz/	Excel/CSV-Export, Sitzungszustand
gui/	PySide6-Oberfläche mit eingebettetem Matplotlib
app.py	Einstiegspunkt

Aufgabe	Modul
Laden der TDMS-Daten, Formaterkennung	bbfmr/io/tdms_laden.py
interne Datenstruktur	bbfmr/io/datensatz.py
Bestimmung der Resonanzfenster (AutoWindow)	bbfmr/fit/autowindows.py
Einzel-Fit eines Linescans	bbfmr/fit/linescan_fit.py
Bewertungskriterien und Schwellwerte	bbfmr/fit/kriterien.py
Stapelverarbeitung aller Linescans	bbfmr/fit/batch.py
Suszeptibilität und S21-Modell	bbfmr/physik/suszeptibilitaet.py, fitmodell.py
Kittel- und Linienbreiten-Auswertung	bbfmr/physik/kittel_llg.py
physikalische Konstanten	bbfmr/physik/konstanten.py
grafische Oberfläche	bbfmr/gui/

Export (Excel, Projektdateien)

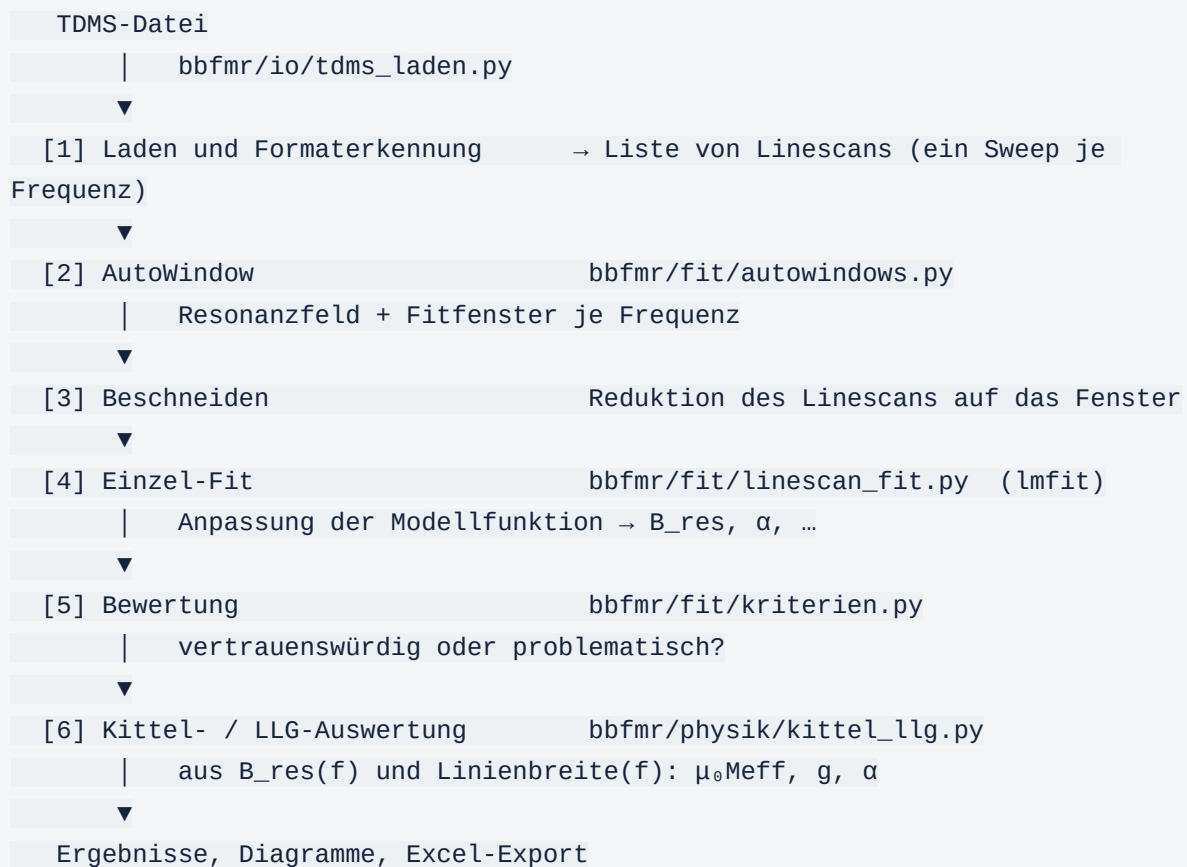
bbfmr/persistenz/

Robustheitsprüfung über reale Daten

tests/autowindow_runner.py

4 Die Auswertekette (Pipeline)

Die Auswertung läuft in einer festen Abfolge ab. Die zentrale Steuerung übernimmt die Funktion `fitte_alle` in `bbfmr/fit/batch.py`. Die Schritte 2–5 werden je Frequenz ausgeführt, die Schritte 1 und 6 auf dem gesamten Datensatz:



Programmatisch genügen wenige Zeilen für eine vollständige Auswertung:

```
from bbfmr.io.tdms_laden import lade_tdms
from bbfmr.fit.batch import fitte_alle

datensatz = lade_tdms("Messung.tdms") # Format wird automatisch erkannt
stapel    = fitte_alle(datensatz)      # AutoWindow + Fit über alle Frequenzen

stapel.index_problematisch()           # Indizes auffälliger Fits
stapel.problem_statistik()             # Häufigkeit der Problemgründe
```

Erweist sich ein einzelner Fit als unbefriedigend, lässt er sich mit veränderten Bandgrenzen, expliziten Startwerten oder vorgegebenem Resonanzfeld gezielt neu rechnen (`fitte_neu`), ohne den übrigen Datensatz erneut zu verarbeiten.

5 Die physikalischen Modelle

Alle Magnetfelder werden konsequent als $\mu_0 \cdot H$ in Tesla geführt, das gyromagnetische Verhältnis $\gamma = g \cdot \mu_B / \hbar$ in $\text{rad}/(\text{s} \cdot \text{T})$. Für $g = 2$ ergibt sich $\gamma \approx 1,7588 \cdot 10^{11} \text{ rad}/(\text{s} \cdot \text{T})$.

Resonanzbedingung (Kittel)

$$\begin{aligned} \text{Out-of-plane (oop):} \quad f &= (\gamma/2\pi) \cdot (\mu_0 H_0 - \mu_0 M_{\text{eff}}) \\ \text{In-plane (ip):} \quad f &= (\gamma/2\pi) \cdot \sqrt{(\mu_0 H_0 + \mu_0 H_u) \cdot (\mu_0 H_0 + \mu_0 H_u + \mu_0 M_{\text{eff}})} \end{aligned}$$

Aus der Anpassung an die Wertepaare (f, B_{res}) folgen $\mu_0 M_{\text{eff}}$ und – bei freigegebenem Parameter – der g-Faktor.

Linienbreite und Gilbert-Dämpfung (LLG)

$$\mu_0 \Delta H(f) = \mu_0 \Delta H_{\text{inh}} + (4\pi/\gamma) \cdot \alpha \cdot f$$

Aus der Steigung dieser Geraden wird die Gilbert-Dämpfung α bestimmt, der Achsenabschnitt $\mu_0 \Delta H_{\text{inh}}$ beschreibt die inhomogene Verbreiterung.

Modellfunktion des Einzel-Fits

$$S_{21}(B) = A \cdot \exp(i\varphi) \cdot \chi(B; B_{\text{res}}, \alpha, \omega, \gamma) + \text{Untergrund}(B)$$

mit der Anregungskreisfrequenz $\omega = 2\pi \cdot f$. Die Suszeptibilität χ ist als **Polder-Suszeptibilität** implementiert; der Untergrund ist eine komplexe, feldabhängige Gerade (Offset und Steigung getrennt für Re und Im). Der Fit erfolgt mit `lmfit` als nichtlineare Ausgleichsrechnung (Levenberg-Marquardt) und passt Real- und Imaginärteil **simultan** an. Freie Parameter:

Parameter	Bedeutung	Schranken
<code>B_res</code>	Resonanzfeld	innerhalb des Fitfensters (verbindlich)
<code>alpha</code>	Gilbert-Dämpfung	[<code>ALPHA_MIN</code> , <code>ALPHA_MAX</code>]

<code>A, phi</code>	Amplitude, Phasenwinkel	<code>phi ∈ [PHI_MIN, PHI_MAX]</code>
<code>off_re/im</code>	Untergrund-Offset	frei
<code>slope_re/ im</code>	Untergrund-Steigung	frei

Die verbindliche Bedingung, dass `B_res` im Fenster liegen muss, koppelt die Fit-Qualität unmittelbar an die korrekte Fensterwahl durch das `AutoWindow`.

Verifizierte Herkunft der Formeln. Die Modellfunktionen sind zeichengenaue Portierungen verbindlicher Quellen und wurden gegen diese geprüft: (1) Dissertation M. Müller (2023), Kap. 2 – Kittel-Gleichungen und Linienbreite im LLG-Bild; (2) Mathematica-Notebook der Suszeptibilitäts-/S21-Fitfunktionen; (3) Messprotokoll inkl. inverser Suszeptibilität (Weiler). Die Zuordnung ist in `docs/physik-und-fit.md` dokumentiert.

6 Das Herzstück: AutoWindow

Der kritischste Schritt für die Zuverlässigkeit ist die automatische Bestimmung des Feldfensters je Frequenz (`bbfmr/fit/autowindows.py`). Sitzt das Fenster falsch, liefert der Fit unbrauchbare Werte – im ungünstigsten Fall ohne Fehlermeldung. Die Resonanz ist häufig nur eine schmale, schwache Struktur auf einem starken, über den Feld-Sweep gekrümmten Untergrund; hinzu kommen feldstationäre Störfeatures (Ripples, Artefakte) und Messrauschen.

Die zentrale Idee: Die echte Resonanz *wandert* mit der Frequenz gemäß der Kittel-Dispersion – `B_res(f)` ist glatt und monoton steigend. Störfeatures sitzen dagegen bei *festen* Feldwerten. Diese Unterscheidung *wandernd* vs. *stationär* trennt die Resonanz von Störungen. Das Verfahren arbeitet in fünf Schritten:

Schritt	Was geschieht
1 — Untergrundabzug	glattes Polynom je Linescan an Re/Im anpassen und abziehen; übrig bleibt das Residuum, in dem die Resonanz hervortritt.
2 — Stationärabzug	bei gemeinsamem Feldgitter: Median über alle Frequenzen je Feldwert abziehen. Hebt die Erkennung auf einem Gitter-Datensatz von ~87 % auf ~95 %.

3 — Kandidat	Ort des Residuum-Maximums je Linescan als Resonanzkandidat, mit robuster Prominenz als Verlässlichkeitsmaß.
4 — Glatte Trasse	gleitende robuste Gerade durch die prominenten Kandidaten; folgt der lokalen Dispersion, unterdrückt Ausreißer.
5 — Fenster	konsistentem Kandidaten wird vertraut, sonst an der Trasse ausgerichtet und auf einen nahen Peak verfeinert. Fensterbreite an die lokale Halbwertsbreite gekoppelt.

Bleibt die Automatik an einem Störfeature hängen, kann die Dispersion in der Oberfläche mit zwei Klicks manuell vorgegeben werden (`fenster_aus_trasse`). Grenzen des Verfahrens (offen ausgewiesen): Messungen nahe der In-plane-Geometrie mit sehr schwachem Signal, antiferromagnetische Proben außerhalb der Kittel-Form sowie Datensätze mit dominanten stationären Hochfeld-Artefakten.

7 Qualitätsbewertung und Verlässlichkeit

Jeder Einzel-Fit wird automatisch als *unauffällig* oder *problematisch* eingestuft (`bbfmr/fit/kriterien.py`). Das Bestimmtheitsmaß R^2 ist hier ungeeignet, weil die Gesamtvarianz vom Offset und vom Untergrund dominiert wird – eine fast gerade Linie erreicht $R^2 \approx 1$, ohne die Resonanz zu beschreiben. Primäres Gütemaß ist daher das **normierte Residuum** (`rmse_norm`). Ein Fit gilt als problematisch, sobald eines der Kriterien zutrifft:

Kriterium	Problemgrund
Residuum <code>rmse_norm</code> über Schwelle oder nicht endlich	„Residuum zu groß“
<code>alpha</code> , <code>phi</code> oder <code>B_res</code> nahe einer Schranke	„... an Grenze“
<code>B_res</code> außerhalb des Fensters	„B_res außerhalb Fenster“
<code>alpha</code> unphysikalisch groß	„alpha unphysikalisch“
kein Konvergenzerfolg / keine Unsicherheiten	„keine Konvergenz“
relative Unsicherheit von <code>B_res</code> zu groß	„B_res-Unsicherheit zu groß“

Damit ist die Bewertung das **Meldeinstrument** des Programms: Ein falsch sitzendes Fenster äußert sich typischerweise in einem dieser Kriterien und wird gemeldet. Die Robustheitsprüfung unterscheidet bewusst zwischen einem *gemeldeten* Problem (zulässig) und einem *still* falsch gesetzten Fenster (der

eigentliche Fehlerfall). Die Schwellwerte sind physikalisch motiviert und werden nicht zur Schönung der Statistik gelockert.

Tests und Robustheits-Harness

Die Korrektheit wird auf zwei Ebenen abgesichert: eine `pytest`-Suite prüft die physikalischen Modelle, das Laden, den Fit und die Bewertung (`python -m pytest -q`); zusätzlich prüft `tests/autowindow_runner.py` das AutoWindow über reale Messdateien des gesamten Datenbestands und weist Fälle, in denen keine zuverlässige Resonanz lokalisierbar ist, offen aus.

8 Installation und Nutzung

```
# virtuelle Umgebung anlegen und aktivieren
python -m venv .venv && source .venv/bin/activate

# bbFMR installieren (editierbar) - mit Oberfläche und Testwerkzeugen
pip install -e ".[gui,test]"
```

Start der grafischen Oberfläche:

```
bbfmr          # grafische Oberfläche
python -m bbfmr.app  # gleichbedeutend
```

Kernabhängigkeiten: `numpy`, `scipy`, `lmfit`, `npTDMS`, `matplotlib`, `pandas`, `openpyxl`; die Oberfläche zusätzlich `PySide6`.

Vollständige Dokumentation. Eine ausführliche Beschreibung von Aufbau, Modellen, Parametern (Tuning) und Fehlersuche befindet sich im Verzeichnis `docs/` (ReadTheDocs-Format, mit `MkDocs` als HTML erzeugbar: `mkdocs serve`). Dieses Dokument ist die kompakte Einführung; `docs/` ist das Nachschlagewerk.

Technischer Steckbrief

Merkmal	Wert
Paketname / Version	<code>bbfmr</code> 0.1.0

Sprache	Python $\geq$ 3.11
Lizenz	MIT
Fit-Engine	<code>lmfit</code> (Levenberg-Marquardt, simultan Re/Im)
Eingabeformat	TDMS (LabVIEW), zwei Varianten, automatisch erkannt
Ausgabe	Materialgrößen, Diagramme, Excel/CSV
Oberfläche	PySide6 mit eingebettetem Matplotlib
Herkunft	Portierung einer LabVIEW-Auswertung nach Python
Dokumentation	<code>docs/</code> (MkDocs / ReadTheDocs-Format)